Introduction

Organizor is the backend API for managing tenant environments. It keeps tenant records in a central catalog database and provisions a separate PostgreSQL database for each tenant.

The current implementation focuses on tenant lifecycle operations:

- creating tenant catalog records;
- onboarding tenants by creating a tenant database and applying migrations through internal operations;
- resolving tenant context for tenant-scoped requests;
- upgrading tenant databases by running migrations;
- soft-deleting tenants before hard deletion;
- exposing health checks, OpenAPI JSON, and Scalar API reference in development.

Main Concepts

Concept	Description
Tenant	A customer or isolated workspace tracked by `Id`, `Name`, `Slug`, `DatabaseName`, lifecycle status, and deletion metadata.
Catalog database	Central PostgreSQL database that stores all tenant records.
Tenant database	PostgreSQL database named from the tenant slug, currently using the `organizor_tenant_{slug}` convention.
Tenant resolution	Middleware step that reads `X-Tenant-Slug`, loads the tenant, and stores the current tenant in `ITenantContext`.
Provisioning workflow	Infrastructure workflow that creates the tenant database and applies tenant migrations.

Project Layout

Project	Responsibility
`Code7Dev.Organizor.Api`	ASP.NET Core host, controllers, DTOs, mappers, middleware, OpenAPI/Scalar, health checks, CORS, and logging setup.
`Code7Dev.Organizor.Application`	Tenant service contracts, tenant models, onboarding, lifecycle, upgrade, and catalog-facing application logic.
`Code7Dev.Organizor.Domain`	Tenant entity and `TenantStatus` enum.
`Code7Dev.Organizor.Infrastructure`	EF Core contexts, repository implementation, PostgreSQL connection string building, database creation/deletion, migration, and tenant context storage.

Generated Reference

DocFX builds API reference pages from the C# projects into the `api` section. Use those pages for class, interface, and method signatures, and use these articles for workflow-level documentation. The site is intended for internal developers and as an outside-IDE AI reference, so implementation details are documented where they affect behavior.

Getting Started

Prerequisites

- .NET SDK 10.
- PostgreSQL available to the API.
- A catalog database connection string named `Catalog`.
- PostgreSQL server settings under the `Postgres` configuration section for tenant database creation and resolution.
- DocFX installed as a .NET tool when building documentation locally.

Run the API

From the repository root:

```
dotnet run --project .\Code7Dev.Organizor.Api\Code7Dev.Organizor.Api.csproj
```

Development launch profiles expose:

Profile	URL
`http`	`http://localhost:5100`
`https`	`https://localhost:7184` and `http://localhost:5100`

On startup the API:

1. configures Serilog;
2. registers catalog and tenant database services;
3. registers application and infrastructure services;
4. seeds or migrates the catalog database;
5. maps controllers, health checks, OpenAPI, and Scalar in development.

Development Endpoints

Endpoint	Purpose
`/health`	ASP.NET Core health checks.
`/openapi/v1.json`	OpenAPI document, available in development.
`/scalar`	Scalar API reference, available in development.

Example tenant catalog request:

```
Invoke-RestMethod `
  -Method Post `
  -Uri "http://localhost:5100/api/v1/tenants" `
  -ContentType "application/json" `
  -Body '{"name":"Acme","slug":"acme","culture":"en-US"}'
```

Build the Documentation

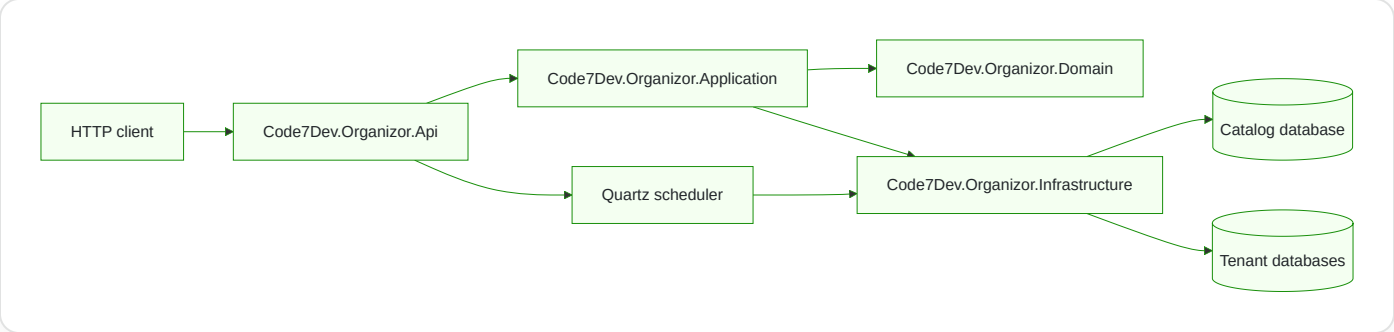
From the repository root:

```
docfx .\docs\docfx.json
```

DocFX writes the generated site to `docs/_site` and generated API metadata to `docs/api`. Both folders are ignored by git and can be rebuilt.

Architecture

Organizor follows a layered structure around tenant management.



API Layer

The API project owns transport concerns:

- controller routes for public tenant catalog operations and internal tenant operations;
- request and response DTOs;
- mapping between API DTOs and application models;
- correlation ID, exception handling, and tenant resolution middleware;
- Serilog request logging;
- CORS, health checks, OpenAPI, and Scalar setup.

The startup flow in `Program.cs` composes the application by registering catalog persistence, tenant persistence, application services, infrastructure services, and API services.

Application Layer

The application project owns use cases and contracts. It defines interfaces for repository access, tenant database management, migration, provisioning, onboarding, upgrades, and lifecycle operations.

Current services:

Service	Responsibility
`TenantService`	Creates catalog tenant records and reads tenants by slug or as a list.
`TenantOnboardingService`	Creates a catalog tenant, runs provisioning, and marks the tenant active or failed.
`TenantLifecycleService`	Soft-deletes tenants and hard-deletes already soft-deleted tenants.
`TenantUpgradeService`	Runs migrations for an existing tenant database.

Domain Layer

The domain project currently contains the tenant entity and status enum. A tenant stores identity, slug, database name, creation date, lifecycle status, provisioning failure reason, and deletion metadata.

Infrastructure Layer

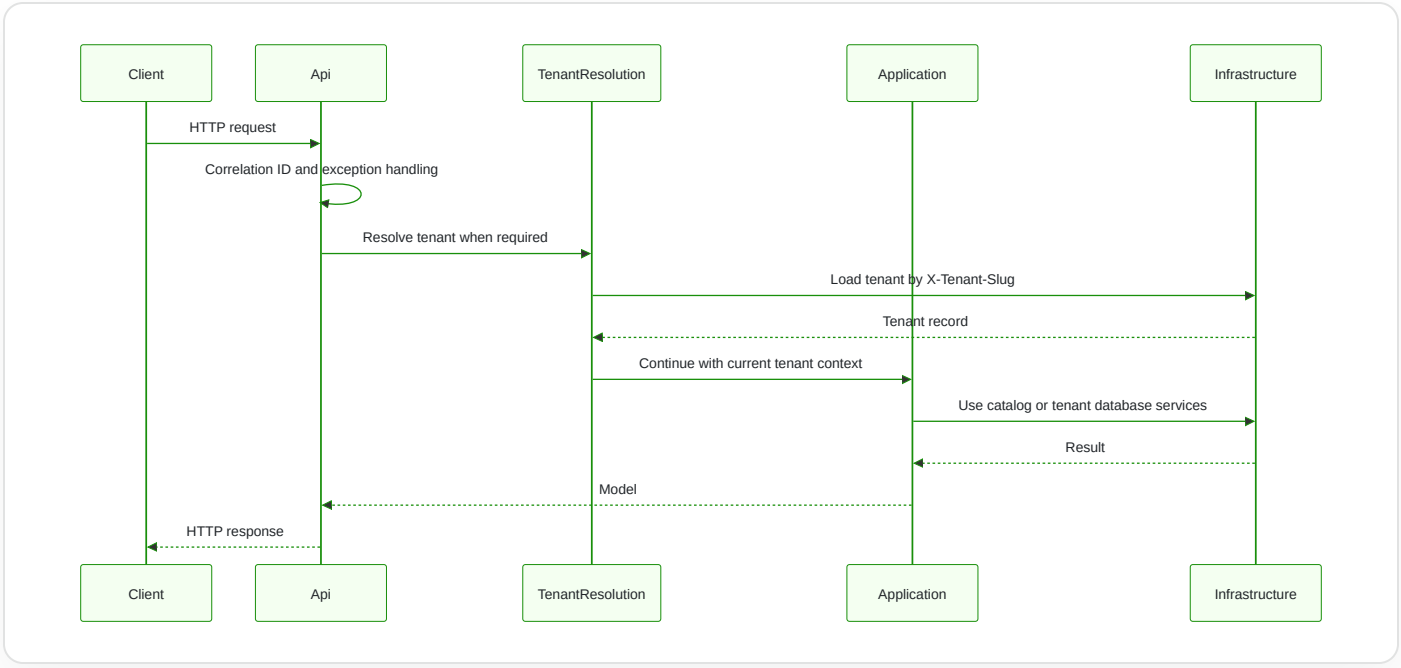
The infrastructure project owns external resources:

- `CatalogDbContext` stores tenant records.
- `OrganizorDbContext` connects to the current tenant database after tenant resolution.
- `TenantRepository` implements catalog tenant persistence.
- `ConnectionStringBuilder` builds PostgreSQL connection strings from configuration.
- `TenantDatabaseManager` creates and drops tenant databases.
- `TenantMigrator` applies EF Core migrations to tenant databases.
- `TenantProvisioningWorkflow` creates a tenant database and applies migrations during onboarding.
- `TenantJobScheduler` schedules tenant-scoped Quartz jobs from catalog tenant records.
- `TenantAwareJobFactory` creates Quartz jobs through dependency injection.
- `SyncTenantJob` runs recurring tenant synchronization work with `ITenantContext` populated from Quartz job data.

Background Jobs

Quartz is registered by the API layer and backed by persistent PostgreSQL storage in the catalog database. On startup, the application schedules a recurring synchronization job for each catalog tenant. Each job carries the tenant ID and slug in Quartz job data, then sets the tenant context before running tenant database work.

Request Pipeline



Tenancy

Organizor uses catalog-driven, database-per-tenant tenancy.

Tenant Catalog

The catalog database stores tenant metadata through `CatalogDbContext`. Each tenant includes:

- `Id`;
- `Name`;
- `Slug`;
- `DatabaseName`;
- `CreatedAt`;
- `Status`;
- `FailureReason`;
- soft-delete metadata.

The database name convention is `organizor_tenant_{slug}`. Slugs are normalized to lowercase before they are stored or used for lookups.

Normal catalog reads exclude soft-deleted tenants. Repository methods can include soft-deleted records when an internal operation explicitly asks for them.

Tenant Statuses

Status	Meaning
`Pending`	Tenant exists in the catalog but provisioning is not complete.
`Active`	Tenant provisioning completed successfully.
`Upgrading`	Reserved for upgrade lifecycle state.
`Failed`	Provisioning failed or an unexpected onboarding error occurred.
`Deleted`	Tenant has been soft-deleted.

Tenant Resolution

`TenantResolutionMiddleware` resolves tenant context for tenant-scoped routes.

It skips platform endpoints such as:

- `/api/admin`;
- `/api/v1/tenants`;
- `/health`;
- `/scalar`;
- `/openapi`;
- root and common static paths.

For tenant-scoped requests, clients must send:

```
X-Tenant-Slug: acme
```

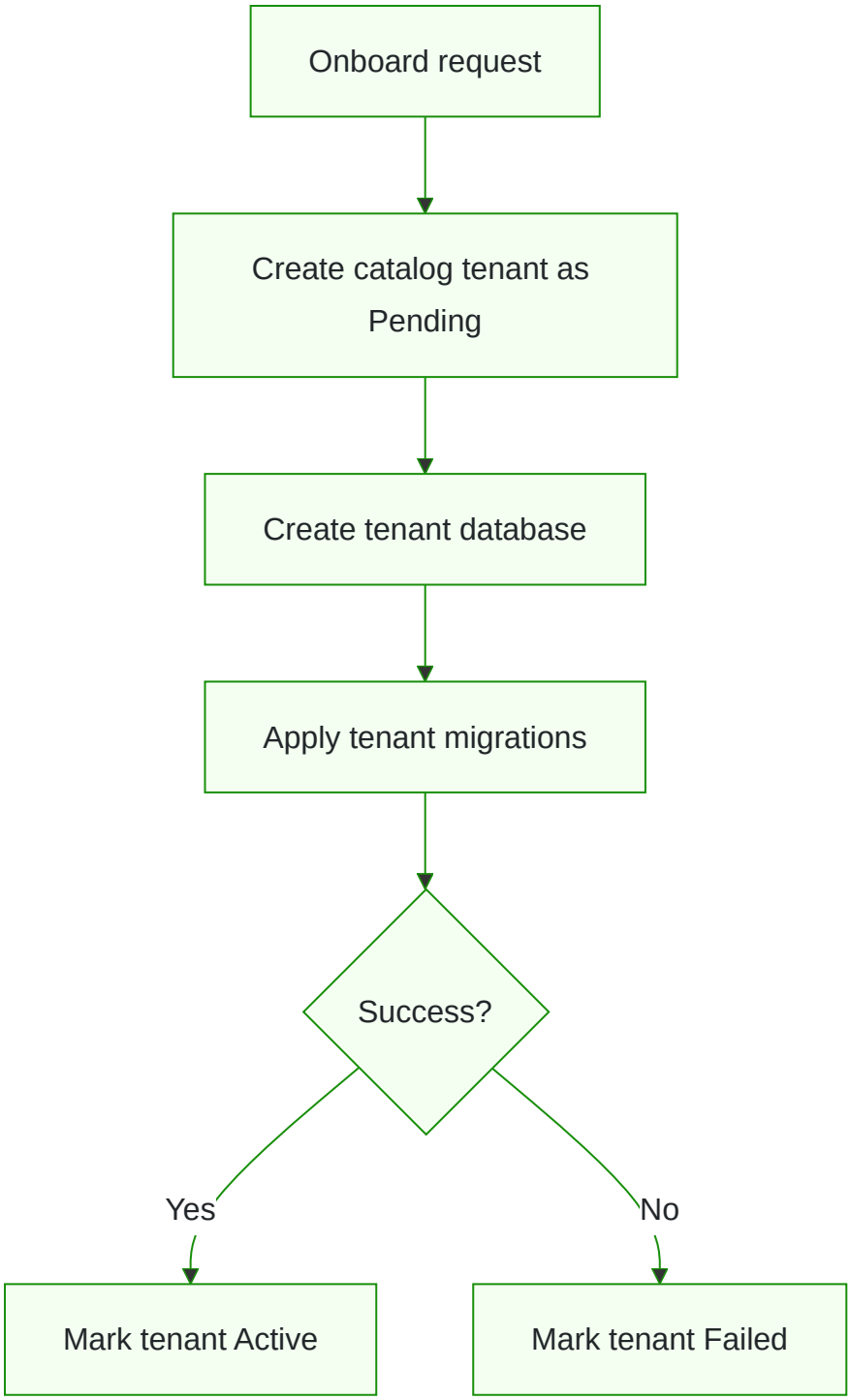
The middleware loads the tenant from the catalog and stores `TenantInfo` in `ITenantContext`. If the header is missing, the response is `400 Bad Request`. If the slug does not match a catalog tenant, the response is `404 Not Found`.

Tenant Database Resolution

`OrganizorDbContext` uses a fallback in-memory database when no tenant has been resolved. This supports startup, Scalar, migrations, and design-time tooling.

When a tenant is resolved, `PerTenantDbResolver` loads the catalog tenant, builds a PostgreSQL connection string for that tenant database, and `OrganizorDbContext` uses that database.

Provisioning Flow



Hard deletion is intentionally guarded: the tenant must be soft-deleted before `TenantLifecycleService` drops the tenant database and removes the catalog record.

API

The public API currently exposes tenant catalog endpoints. Tenant-scoped application endpoints should include the `X-Tenant-Slug` header unless the path is explicitly skipped by tenant resolution.

Internal tenant administration endpoints are documented separately in Internal Operations.

Tenant Catalog

Create Tenant

```
POST /api/v1/tenants
Content-Type: application/json

{
  "name": "Acme",
  "slug": "acme",
  "culture": "en-US"
}
```

Creates a catalog tenant record with `Pending` status and returns tenant metadata. This route does not provision a tenant database.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
  "slug": "acme",
  "databaseName": "organizor_tenant_acme",
  "createdAt": "2026-05-15T10:00:00Z"
}
```

Possible responses:

Status	Meaning
`200 OK`	Tenant catalog record was created.
`500 Internal Server Error`	Duplicate slug or unexpected server error. See Errors and Observability.

Get Tenant By Slug

```
GET /api/v1/tenants/acme
```

Returns a tenant by slug, or `404 Not Found` when no tenant exists. Soft-deleted tenants are excluded.

List Tenants

```
GET /api/v1/tenants
```

Returns all catalog tenants visible through the tenant repository. Soft-deleted tenants are excluded.

Platform Endpoints

Endpoint	Purpose
`/health`	Public health check endpoint.
`/openapi/v1.json`	OpenAPI document in development.
`/scalar`	Scalar API reference in development.

Local API Reference

When the API runs in development:

- OpenAPI JSON is available at `/openapi/v1.json`.
- Scalar is available at `/scalar`.

The DocFX API navigation tab contains the generated C# reference for controllers, DTOs, services, interfaces, middleware, and persistence types.

Database Provisioning

Organizor uses one catalog database and one PostgreSQL database per tenant.

Catalog Database

`CatalogDbContext` stores tenant records in the `tenants` table.

Important constraints:

Field	Constraint
`Name`	Required, max length 200.
`Slug`	Required, max length 100, unique index.
`DatabaseName`	Required, max length 200.
`IsDeleted`	Required, defaults to `false`.
`DeletionReason`	Optional, max length 500.

On startup, the API checks whether the catalog database exists, creates it when needed, and applies catalog migrations.

Tenant Database Naming

Tenant databases use:

```
organizor_tenant_{slug}
```

Example:

```
organizor_tenant_acme
```

Slugs are normalized to lowercase before tenant records are written.

PostgreSQL Requirements

The configured PostgreSQL user must be able to:

- connect to the admin `postgres` database;
- create tenant databases;
- drop tenant databases during hard deletion;
- create extensions in tenant databases.

Tenant database creation currently uses:

```
CREATE DATABASE "{databaseName}" OWNER = postgres ENCODING = 'UTF8';
```

Each tenant database is initialized with:

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "citext";
```

Hard deletion uses:

```
DROP DATABASE IF EXISTS "{databaseName}" WITH (FORCE);
```

Migration Flow

`TenantMigrator` builds a tenant-specific connection string, checks pending migrations on `OrganizorDbContext`, applies them, and returns:

Field	Meaning
`Database`	Tenant database name.
`Success`	Whether migrations completed.
`AppliedMigrations`	Count of migrations pending before execution.
`Duration`	Migration runtime.

Field	Meaning
`Error`	Error message when migration fails.

Keycloak Setup

This document describes the Keycloak configuration for the Organizor development environment. It includes realm setup, API/UI clients, tenant groups, claim mapping, and user/role setup.

Realm Setup

- Open Keycloak Admin Console.
- Realm dropdown -> **Create Realm**.
- Name: `organizor-dev`.
- Save.

API Client (`organizor-api`)

Settings:

- Client type: OpenID Connect
- Client ID: `organizor-api`
- Name: Organizor API

Required values:

- Client authentication: **OFF**
- Authorization: OFF
- Standard flow: OFF
- Direct access grants: **ON**
- Implicit flow: OFF
- Service accounts roles: **ON**

Credentials:

- Open the **Credentials** tab.
- Copy the **Client Secret**.
- Store it in development tooling that requests tokens.

The API currently validates bearer tokens only; it does not read the client secret from application configuration. The client secret is only needed by tooling or services that request tokens with the `client_credentials` flow.

UI Client (`organizor-ui`)

Settings:

- Client type: OpenID Connect
- Client ID: `organizor-ui`
- Name: Organizor UI

Required values:

- Client authentication: OFF
- Authorization: OFF
- Standard flow: **ON**
- Direct access grants: OFF
- Implicit flow: OFF
- Service accounts roles: OFF

Development redirect URIs:

- `https://localhost:5001/*`
- `https://localhost:5002/*`
- `http://localhost:5001/*`
- `http://localhost:5002/*`

Web origins:

- `*`

Tenant Groups

Group structure:

- `/tenants`
- `/tenants/acme`
- `/tenants/acme/users`

- `/tenants/acme/admins`

Each tenant has:

- `/users` group
- `/admins` group

Tenant Membership Claim Mapper

Location:

- Clients -> `organizor-api`
- Client scopes -> `organizor-api-dedicated`
- Mappers -> Create

Values:

- Name: `tenant-membership`
- Mapper type: **Group Membership**
- Token Claim Name: `tenant`
- Full group path: **ON**
- Add to ID token: ON
- Add to access token: ON
- Add to userinfo: ON

User Setup

Platform admin:

- Username: `admin@organizor.dev`
- Password: `Admin123!`
- Assign realm role: `organizor-admin`
- No tenant groups assigned

Tenant admin:

- Username: `admin@acme.dev`
- Password: `Admin123!`
- Assign realm role: `organizor-user`

- Assign group: `/tenants/acme/admins`

Tenant user:

- Username: `user@acme.dev`
- Password: `Admin123!`
- Assign realm role: `tenant-user`
- Assign group: `/tenants/acme/users`

Token Testing

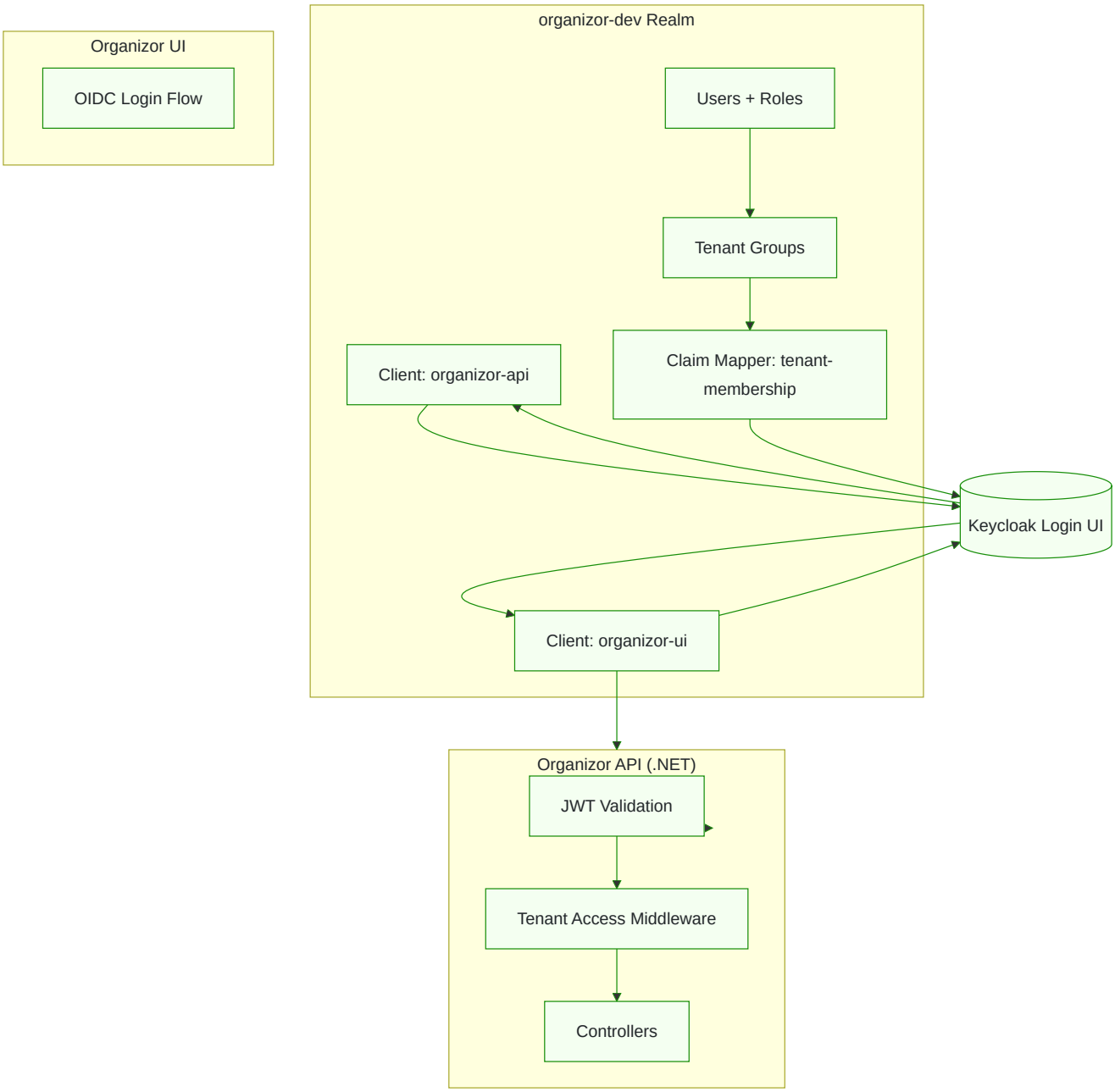
Token endpoint:

- `http://localhost:8080/realms/organizor-dev/protocol/openid-connect/token`

Example tenant claim:

```
"tenant": ["/tenants/acme/admins"]
```

Keycloak Architecture Diagram


Organizor UI
OIDC Login Flow
organizor-dev Realm
Users + Roles
Tenant Groups
Client: organizor-api
Claim Mapper: tenant-membership
Keycloak Login UI
Client: organizor-ui
Organizor API (.NET)
JWT Validation
Tenant Access Middleware
Controllers

API Authentication

Overview

This document explains how the Organizor API authenticates requests using Keycloak-issued JWT tokens.

JWT Validation

The API validates tokens using the Keycloak realm:

- Authority: `http://localhost:8080/realms/organizor-dev`
- Audience validation: disabled for the current development setup
- Token type: Bearer

The authentication middleware:

- validates the token signature;
- validates the issuer;
- validates token expiration;
- populates `HttpContext.User`.

Audience validation is intentionally disabled in code while the Keycloak client and token shape are still being finalized. Once the API has a stable audience claim, enable audience validation and document the expected value here.

Authentication Flow

1. Frontend logs in via Keycloak.
2. Keycloak issues a JWT.
3. Frontend sends `Authorization: Bearer <token>`.
4. Frontend also sends `X-Tenant-Slug: acme` for tenant-scoped routes.
5. API validates the JWT.
6. `[Authorize]` attributes enforce authentication where applied.

Service Accounts

The `organizor-api` client can use the `client_credentials` flow for internal API-to-API communication. Service account tokens are identified by the `client_id` claim:

```
"client_id": "organizor-api"
```

Service accounts bypass tenant access checks.

WhoAmI Endpoint

Example response:

```
{
  "user": "admin@acme.dev",
  "claims": [
    { "type": "sub", "value": "..." },
    { "type": "tenant", "value": "/tenants/acme/admins" },
    { "type": "role", "value": "organizor-user" }
  ]
}
```

Error Codes

- 401 Unauthorized -> Missing or invalid token.
- 403 Forbidden -> Token is valid, but the user is not allowed.
- 400 Bad Request -> Missing or invalid tenant header on tenant-scoped routes.

Test Request

```
GET http://localhost:5100/api/auth/whoami
Authorization: Bearer <token>
```

Tenant Authorization

Overview

This document describes how the Organizor API resolves tenant context and enforces tenant-level access control.

Tenant Resolution

Tenant-scoped routes must include:

```
X-Tenant-Slug: acme
```

The tenant resolution middleware skips platform and infrastructure routes, including:

- `/api/admin`
- `/api/v1/tenants`
- `/api/auth`
- `/health`
- `/openapi`
- `/scalar`
- `/favicon`
- `/robots`

Resolution steps:

1. Middleware reads `X-Tenant-Slug`.
2. Middleware looks up the tenant in the catalog database.
3. Middleware sets `tenantContext.Current`.
4. Middleware logs tenant metadata.

JWT Tenant Claim

Tenant membership is represented by Keycloak group claims:

```
"tenant": ["/tenants/acme/admins"]
```

Tenant Validation Logic

Current and planned rules:

- If JWT contains tenant claims, the header must match.
- If tenant claims mismatch, the request returns 403 Forbidden.
- If no tenant claim exists, superadmin access can bypass tenant membership checks.
- Service accounts bypass tenant access checks.
- DB-level access is enforced through `ITenantAccessService`.

Tenant claim matching and role policies are still being implemented. The current middleware resolves the tenant and delegates access checks to `ITenantAccessService`.

Role-Based Authorization

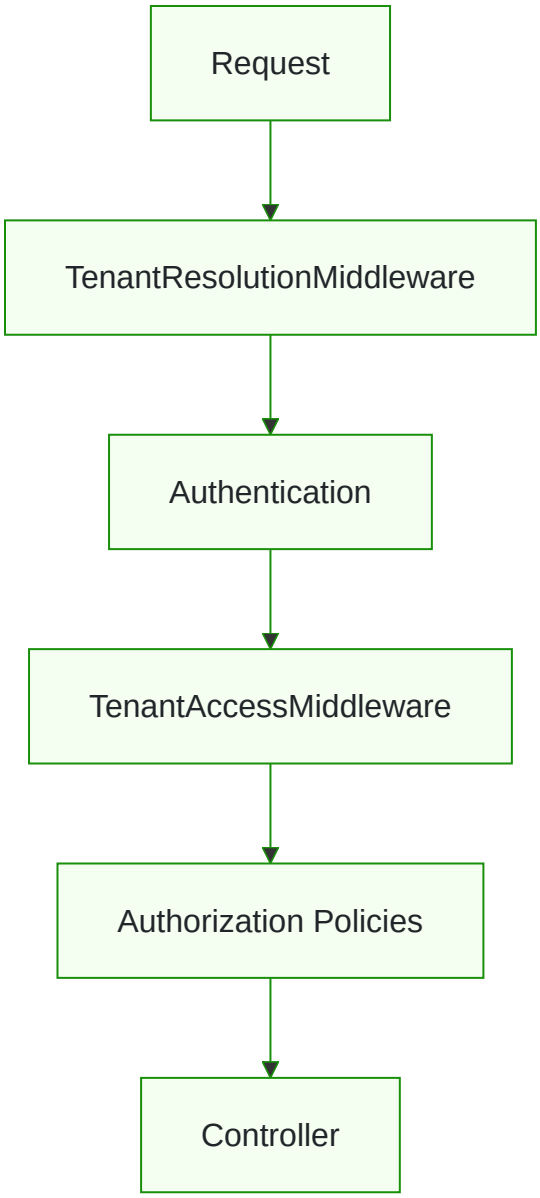
Realm roles:

- `organizor-admin` -> platform admin
- `organizor-user` -> tenant admin
- `tenant-user` -> tenant user

Example:

```
[Authorize(Roles = "organizor-admin")]
```

Middleware Chain



Example Request

```
GET /api/some-tenant-scoped-route
Authorization: Bearer <token>
X-Tenant-Slug: acme
```

Configuration

Organizor reads standard ASP.NET Core configuration from `appsettings.json`, environment-specific files, user secrets, and environment variables.

Catalog Connection String

`CatalogDbContext` expects a connection string named `Catalog`.

Example shape:

```
{
  "ConnectionStrings": {
    "Catalog":
"Host=localhost;Port=5432;Database=organizor_catalog;Username=postgres;Password
=postgres"
  }
}
```

Tenant PostgreSQL Settings

`ConnectionStringBuilder` builds tenant and admin connection strings from the `Postgres` section.

```
{
  "Postgres": {
    "Host": "localhost",
    "Port": "5432",
    "User": "postgres",
    "Password": "postgres"
  }
}
```

The admin connection uses the `postgres` database. Tenant connections use the tenant database name stored in the catalog.

Quartz

Quartz uses the `Catalog` connection string for its persistent store. The same catalog database stores tenant records and Quartz scheduler tables.

The API registration configures:

- PostgreSQL-backed Quartz persistence;
- Newtonsoft JSON serialization for persisted job data;
- clustering support;
- hosted service startup for Quartz background processing.

Tenant job scheduling is not enabled on application startup yet. `TenantJobScheduler` is available for future real tenant jobs, but the startup scheduling block is currently commented out until those jobs exist.

Logging

Serilog is configured in `appsettings.json`.

Current sinks:

- console;
- rolling file logs under `logs/log-.txt`.

Log output includes tenant and request correlation properties when available:

- `Id`;
- `Slug`;
- `DatabaseName`;
- `CorrelationId`.

CORS

The API selects a CORS policy by environment:

- `DevCors` in development allows any origin, header, and method.
- `ProdCors` outside development must be configured with explicit allowed origins before browser clients can call the API.

The current production policy has no origins configured. Treat that as a deployment checklist item, not a complete production setting.

OpenAPI and Scalar

OpenAPI and Scalar endpoints are mapped only in development. This keeps the production surface smaller while retaining local API exploration during development.

Localization

Organizor provides multilingual support for error responses using a simple, predictable, and API-friendly localization pipeline.

Supported Languages

Organizor currently supports:

- `nl` — Dutch
- `en` — English (fallback)

Additional languages can be added by creating new resource files in the `/Resources` folder.

How Localization Works

Localization is driven by the standard `Accept-Language` HTTP header.

Example:

```
Accept-Language: nl
```

The API resolves the culture for the request and uses it to translate error messages. If a translation is missing, the system falls back to English (`en`).

Key Principles

- Application services return **error codes only** (e.g. `TENANT_NOT_FOUND`).
- The API middleware maps these codes to localized resource strings.
- No hardcoded messages exist in the domain or application layers.
- Localization is deterministic and client-controlled.

Culture Resolution

Organizor uses the following resolution order:

- Accept-Language header
- Fallback to English (en)

This keeps behavior predictable and client-controlled.

Error Code Translation

Each error code corresponds to a resource key.

Examples:

- TENANT_NOT_FOUND
- TENANT_ALREADY_EXISTS
- UNEXPECTED_ERROR

These keys exist in:

- /Resources/SharedResources.en.resx
- /Resources/SharedResources.nl.resx

The middleware retrieves the localized string and injects it into the RFC7807 error response.

Example Request and Response

Request

```
GET /api/v1/tenants/unknown-id
Accept-Language: nl
```

Response

```
{
  "type": "https://httpstatuses.com/404",
  "title": "Tenant niet gevonden",
  "status": 404,
  "errorCode": "TENANT_NOT_FOUND",
```

```
  "traceId": "00-abc123..."
}
```

The errorCode remains stable and language-independent. The title is localized based on the request culture.

Adding New Translations

To add a new language:

Create a new .resx file, e.g.:

- SharedResources.fr.resx Add the same error keys as in the English file

Provide translated values

Restart the API

Errors and Observability

Correlation IDs

Every request passes through `CorrelationIdMiddleware`.

Clients can provide a correlation ID:

```
X-Correlation-ID: request-123
```

If the header is missing, the API generates a GUID. The final value is returned in the response header:

```
X-Correlation-ID: 2ad71e7a-13cb-4ef8-a4c7-f3a37ad9dc71
```

The same value is pushed into Serilog context as `CorrelationId`.

Error Shape

Unhandled exceptions are returned as RFC 7807-style problem details:

```
{
  "type": "https://httpstatuses.com/500",
  "title": "An unexpected error occurred.",
  "status": 500,
  "detail": "Exception message",
  "instance": "/api/v1/tenants",
  "correlationId": "2ad71e7a-13cb-4ef8-a4c7-f3a37ad9dc71"
}
```

The response content type is:

```
application/problem+json
```

> **NOTE**
>
> The current middleware includes exception messages in `detail`. Treat that as useful during development, but review it before exposing production environments.

Tenant Resolution Errors

Tenant-scoped requests require:

```
X-Tenant-Slug: acme
```

Missing tenant slug returns `400 Bad Request` with a plain text response. Unknown tenant slug returns `404 Not Found` with a plain text response.

Log Context

Serilog output is configured to include:

Property	Source
`CorrelationId`	Request header or generated GUID.
`Id`	Resolved tenant ID.
`Slug`	Resolved tenant slug.
`DatabaseName`	Tenant database selection when available.
`SourceContext`	Logger source in file logs.

Internal Operations

This page documents internal routes and workflows for developers and operators.

Onboard Tenant

```
POST /api/admin/tenants/onboard
Content-Type: application/json

{
  "name": "Acme",
  "slug": "acme"
}
```

The onboarding flow:

1. normalizes the slug to lowercase;
2. creates a catalog tenant with `Pending` status;
3. creates the tenant database using `organizor_tenant_{slug}`;
4. initializes PostgreSQL extensions;
5. applies tenant migrations;
6. marks the tenant `Active` on success or `Failed` on provisioning failure.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
  "slug": "acme",
  "databaseName": "organizor_tenant_acme"
}
```

Upgrade Tenant

```
POST /api/admin/tenants/{id}/upgrade
```

Runs pending migrations for the tenant database.

Example response:

```
{
  "database": "organizor_tenant_acme",
  "success": true,
  "appliedMigrations": 2,
  "duration": "00:00:01.2340000",
  "error": null
}
```

Soft Delete Tenant

```
DELETE /api/admin/tenants/{id}
Content-Type: application/json

{
  "reason": "Customer cancellation"
}
```

Soft deletion marks the tenant as deleted, stores deletion metadata, and sets status to `Deleted`. The tenant database is not dropped.

Hard Delete Tenant

```
POST /api/admin/tenants/{id}/hard-delete
```

Hard deletion requires the tenant to already be soft-deleted. It drops the tenant database and removes the catalog record.

Scheduled Tenant Jobs

Organizor uses Quartz for recurring background work. During startup, `Program.cs` resolves `TenantJobScheduler` and calls `ScheduleAllTenantsAsync`.

The scheduler:

1. loads all tenants from the catalog database;
2. creates a durable `SyncTenantJob` with job key `sync-{tenantId}` in the `tenant-jobs` group;
3. stores `TenantId` and `TenantSlug` in the Quartz job data map;
4. creates a trigger named `sync-{tenantId}-trigger` in the `tenant-triggers` group;
5. schedules the job when it is new, or replaces and reschedules it when it already exists.

Current trigger cadence:

```
0/10 * * * * ?
```

That Quartz cron expression runs the tenant synchronization job every 10 seconds.

`SyncTenantJob` reads the tenant data from the job data map, sets `ITenantContext`, and then runs tenant-scoped work through `OrganizorDbContext`. The current implementation logs execution and contains placeholder synchronization work.

Quartz persistence is stored in the catalog database and uses the PostgreSQL tables added by the `AddQuartzTables` migration. The API configures Quartz with the `Catalog` connection string, JSON serialization, and clustering enabled.

Documentation Deployment

The repository contains a GitHub Actions workflow at `.github/workflows/docfx-deploy.yml`.

On pushes to `master`, it:

1. checks out this repository;
2. installs DocFX;
3. builds `docs/docfx.json`;
4. checks out the public documentation repository;
5. copies `docs/_site` into that repository;
6. commits and pushes the generated documentation.

The workflow uses the `DOCS_DEPLOY_KEY` secret to push to the public docs repository.

Sitemap and Crawlers

DocFX generates `docs/_site/sitemap.xml` from the `build.sitemap.baseUrl` value in `docs/docfx.json`.

The documentation source includes `robots.txt`, which is copied into the generated site and points crawlers to the sitemap.